Current	Result	Size	Time	Cycles	GPU	SM Frequency	Attributes
1673 - circuit_kernel_batch_plain_stack	(98, 1, 1)x(128, 1, 1)	1.15 ms	1,696,642	0 - NVIDIA GeForce RTX 2060 SUPER	1.47 Ghz	[32]x[6] python3.11	

Summary | Statistics | Source | Context | Comments | Raw | Session

Compare | Tools | View | Export

GPU Speed Of Light Throughput

GPU Throughput Chart

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric. Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	31.65	Duration [ms]	1.15
Memory Throughput [%]	21.78	Elapsed Cycles [cycle]	1696642
L1/TEX Cache Throughput [%]	24.75	SM Active Cycles [cycle]	1596554.85
L2 Cache Throughput [%]	4.72	SM Frequency [Ghz]	1.47
DRAM Throughput [%]	1.72	DRAM Frequency [Ghz]	6.89

Small Grid This kernel grid is too small to fill the available resources on this device, resulting in only 0.36 full waves across all SMs. Look at Launch Statistics for more details.

Key Performance Indicators

Roofline Analysis The ratio of peak float (FP32) to double (FP64) performance on this device is 32:1. The workload achieved close to 0% of this device's FP32 peak performance and 7% of its FP64 peak performance. See the Roofline charts for more details on roofline analysis.

GPU Throughput

PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [cycle]	20500	# Pass Groups 1
Maximum Buffer Size [Kb/sm]	524.29	-

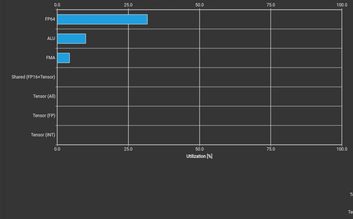
Compute Workload Analysis

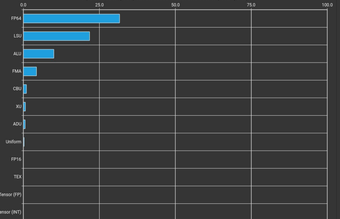
Pipe Utilization (Elapsed Cycles)

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	0.49	Issue Slots Busy [%]	31.65
Executed Ipc Active [inst/cycle]	0.50	Issue Slots Busy [%]	12.56
Issued Ipc Active [inst/cycle]	0.53		

Balanced FP64 is the highest-utilized pipeline (31.6%) based on elapsed cycles in the workload, taking into account the rates of its different instructions. It executes 64-bit floating point operations. It is well-utilized, but should not be a bottleneck.





Memory Workload Analysis

Memory Chart

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/s]	7.60	Mem Busy [%]	12.38
L1/TEX Hit Rate [%]	81.55	Max Bandwidth [%]	21.78
L2 Hit Rate [%]	98.10	Mem Pipes Busy [%]	21.78
Local Memory Spilling Requests	0	Shared Memory Spilling Requests	0
Local Memory Spilling Request Overhead [%]	0	Shared Memory Spilling Request Overhead [%]	0
L2 Sector Promotion Misses [%]	13.62		

Local Memory Usage Est. Speedup: 25.14% This workload accesses 16 local memory sectors, accounting for 11.36% of all sectors requested in L1TEX. Local memory refers to a memory space that is private to each thread. Although it is logically distinct, local memory resides in global memory. This can lead to higher memory latency compared to the usage of registers or shared memory. Consider moving local memory to shared memory or registers if possible.
0.00% of all local loads and 31.93% of all local stores spill into the L2 cache, accounting for 0.90% of all sectors requested in L2.
One common reason for the use of local memory is arrays being too large to fit into registers or not being indexed using compile time constants (see the 'Local Memory' section of the CUDA Programming Guide). For more advice on how to reduce the usage of local memory refer to the GPU documentation.

Key Performance Indicators

L1TEX Global Load Access Pattern Est. Speedup: 21.68% The memory access pattern for global loads to L1TEX might not be optimal. On average, only 4.1 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Address Counters section for uncoalesced global loads.

Key Performance Indicators

L1TEX Global Store Access Pattern Est. Speedup: 7.65% The memory access pattern for global stores to L1TEX might not be optimal. On average, only 22.1 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Address Counters section for uncoalesced global stores.

Key Performance Indicators

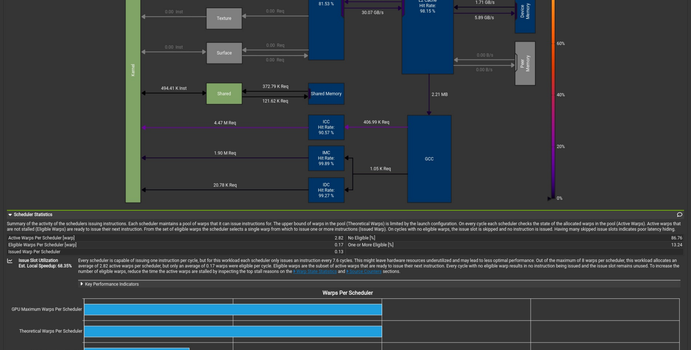
L1TEX Local Load Access Pattern Est. Speedup: 18.35% The memory access pattern for local loads from L1TEX might not be optimal. On average, only 8.3 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Address Counters section for uncoalesced local loads.

Key Performance Indicators

L1TEX Local Store Access Pattern Est. Speedup: 23.91% The memory access pattern for local stores to L1TEX might not be optimal. On average, only 1.1 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Address Counters section for uncoalesced local stores.

Key Performance Indicators

Memory Chart



Scheduler Statistics

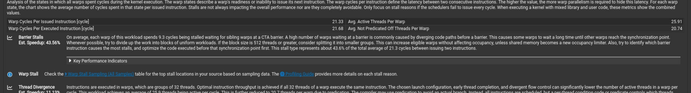
Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	2.82	No Eligible [%]	86.76
Eligible Warps Per Scheduler [warp]	0.17	One or More Eligible [%]	13.24
Issued Warp Per Scheduler	0.13		

Issue Slot Utilization Every scheduler is capable of issuing one instruction per cycle, but for this workload each scheduler only issues an instruction every 7.6 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 8 warps per scheduler, this workload allocates an average of 2.82 active warps per scheduler, but only an average of 0.17 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, reduce the time the active warps are stalled by inspecting the top stall reasons on the Warp State chart and Source Counters sections.

Key Performance Indicators

Warps Per Scheduler



Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined results.

Warp Cycles Per Issued Instruction [cycle]	21.33	Avg. Active Threads Per Warp	25.91
Warp Cycles Per Executed Instruction [cycle]	21.68	Avg. Not Predicated Off Threads Per Warp	20.74

Barrier Stalls Est. Speedup: 43.56% On average, each warp of this workload spends 9.3 cycles being stalled waiting for sibling warps at a CTA barrier. A high number of warps waiting at a barrier is commonly caused by diverging code paths before a barrier. This causes some warps to wait a long time until other warps reach the synchronization point. Whenever possible, try to divide up the work into blocks of uniform workloads. If the block size is 512 threads or greater, consider splitting it into smaller groups. This can increase eligible warps without affecting occupancy, unless shared memory becomes a new occupancy limiter. Also, try to identify which barrier instruction causes the most stalls, and optimize the code executed before that synchronization point first. This stall type represents about 43.6% of the total average of 21.3 cycles between issuing two instructions.

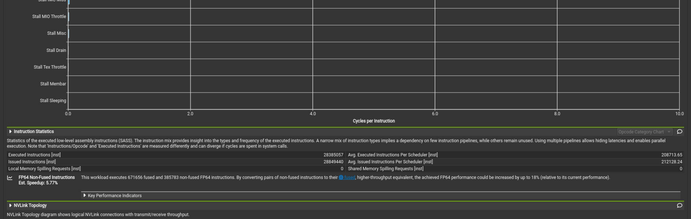
Key Performance Indicators

Warp Stall Check the Warps Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The Profiling Guide provides more details on each stall reason.

Thread Divergence Est. Speedup: 11.15% Instructions are executed in warps, which are groups of 32 threads. Optimal instruction throughput is achieved if all 32 threads of a warp execute the same instruction. The chosen launch configuration, early thread completion, and divergent flow control can significantly lower the number of active threads in a warp per cycle. This workload achieves an average of 20.9 threads being active per cycle. This is further reduced to 20.7 threads per warp due to predication. The compiler may use predication to avoid an actual branch. Instead, all instructions are scheduled, but a per-thread condition code or predicate controls which threads execute the instructions. Try to avoid different execution paths within a warp where possible.

Key Performance Indicators

Warp State (All Cycles)



Instruction Statistics

Opcode Category Chart

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	20385507	Avg. Executed Instructions Per Scheduler [inst]	208713.65
Issued Instructions [inst]	2084648	Avg. Issued Instructions Per Scheduler [inst]	21210.24
Local Memory Spilling Requests [inst]	0	Shared Memory Spilling Requests [inst]	0

FP64 Heavy Kernel Instructions Est. Speedup: 5.77% This workload executes 671656 fused and 385783 non-fused FP64 instructions. By converting pairs of non-fused instructions to their DFMA higher-throughput equivalent, the achieved FP64 performance could be increased by up to 18% (relative to its current performance).

Key Performance Indicators

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	98	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	64	Static Shared Memory Per Block [Kbyte/block]	4.61
Block Size	128	Dynamic Shared Memory Per Block [Byte/block]	0
Threads [thread]	12544	Driver Shared Memory Per Block [byte/block]	0
Waves Per SM	0.36	Shared Memory Configuration Size [Kbyte]	65.54
Uses Green Context	0	Stack Size	1024
# SMs [SM]	34	# TPCs	17
Enabled TPC IDs	all		

Occupancy

% Occupancy Graphs

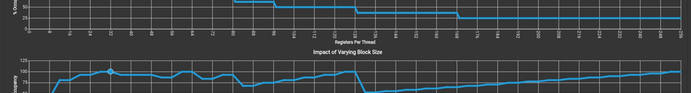
Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	8
Theoretical Active Warps per SM [warps]	32	Block Limit Shared Mem [block]	13
Achieved Occupancy [%]	35.29	Block Limit Warps [block]	8
Achieved Active Warps Per SM [warps]	11.32	Block Limit SM [block]	16

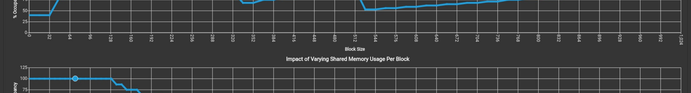
Achieved Occupancy Est. Speedup: 64.61% The difference between calculated theoretical (100.0%) and measured achieved occupancy (35.4%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the Profiling Guide Occupancy Section for more details on optimizing occupancy.

Key Performance Indicators

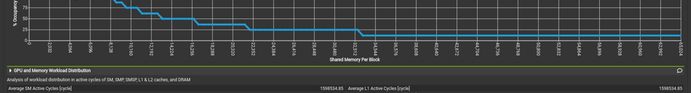
Impact of Varying Register Count Per Thread



Impact of Varying Block Size



Impact of Varying Shared Memory Usage Per Block



GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	1596554.85	Average L1 Active Cycles [cycle]	1596554.85
Average L2 Active Cycles [cycle]	737904.25	Average SMSP Active Cycles [cycle]	1602091.42
Average DRAM Active Cycles [cycle]	136379.50	Total SM Elapsed Cycles [cycle]	57429746
Total L1 Elapsed Cycles [cycle]	57429196	Total L2 Elapsed Cycles [cycle]	51883328
Total SMSP Elapsed Cycles [cycle]	229718984	Total DRAM Elapsed Cycles [cycle]	63338496

L2 Slices Workload Imbalance Est. Speedup: 5.74% One or more L2 Slices have a much higher number of active cycles than the average number of active cycles. Maximum instance value is 12.60% above the average, while the minimum instance value is 6.64% below the average.

Key Performance Indicators

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	2973259	Branch Efficiency [%]	98.31
Branch Instruction Ratio [%]	0.10	Avg. Divergent Branches [branches]	221.90

Uncoalesced Global Accesses Est. Speedup: 4.18% This workload has uncoalesced global accesses resulting in a total of 633480 excessive sectors (9% of the total 6905053 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide has additional information on reducing uncoalesced device memory accesses.

Key Performance Indicators

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
0x000b00440 in circuit_kernel_batch_plain_stack	35,130	5
0x000b00450 in circuit_kernel_batch_plain_stack	35,130	5
0x000b00d30 in circuit_kernel_batch_plain_stack	35,130	5
0x000b00d70 in circuit_kernel_batch_plain_stack	35,130	5
0x000b00d60 in circuit_kernel_batch_plain_stack	35,130	5

Follow the *sass* outputs to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel.
You could also disable Source analysis to focus on selected performance aspects and make profiling faster.